Aufgabe 1: Aktives Warten nach Peterson

Petersons Algorithmus für wechselseitigen Ausschluss – Formelsammlung & Übersicht

1. Zielsetzung & Problemstellung:
Garantie des wechselseitigen Ausschlusses (*Mutual Exclusion*) zweier konkurrierender Prozesse/Threads (P_0 und P_1) im User Space ohne spezielle atomare Hardware-Befehle (wie TSL, XCHG oder CAS).

2. Gemeinsame Variablen (*Shared State*):

- `bool interested[2] = {false, false};` – Signalisiert die Absicht eines Prozesses, den kritischen Abschnitt zu betreten.
- `int turn = 0;` – Höflichkeitsvariable zum Symmetriebruch: Bestimmt, wer im Konfliktfall warten muss.

3. Protokoll im Überblick:

Eintritt: Prozess P_0	**Eintritt: Prozess P_1**
`interested[0] = true;`	`interested[1] = true;`
`turn = 0; // überlässt P1 den Vortritt`	`turn = 1; // überlässt P0 den Vortritt`
`while (interested[1] && turn == 0); // warten`	`while (interested[0] && turn == 1); // warten`
`// Kritischer Abschnitt`	`// Kritischer Abschnitt`
`interested[0] = false; // Austritt`	`interested[1] = false; // Austritt`

Kernprinzip: Jeder Prozess setzt bei Eintritt `turn` auf sich selbst (Höflichkeitsprinzip). Wer als letzter schreibt, muss warten!

Simulator ↗

1.) Standard-Ablauf & Zeitlich versetzter Start

Prozess P_0 betritt zum Zeitpunkt t_0 als erster den kritischen Abschnitt. Während P_0 rechnet, möchte Prozess P_1 zum Zeitpunkt t_1 ebenfalls eintreten und ruft das Eintrittsprotokoll auf.

Aufgaben:

1. Welche Werte haben die Variablen `interested[0]`, `interested[1]` und `turn` zum Zeitpunkt t_1?
2. Warum tritt Prozess P_1 in die `while`-Warteschleife ein?
3. Was passiert genau, wenn Prozess P_0 den kritischen Abschnitt verlässt?

Simulator ↗

2.) Symmetriebruch bei zeitgleichem Eintreffen

Nehmen Sie an, Prozess P_0 und Prozess P_1 rufen exakt zeitgleich das Eintrittsprotokoll auf und beide setzen ihr jeweiliges `interested`-Flag auf `true`.

Aufgaben:

1. Warum entsteht hierbei **keine** Verklemmung (*Deadlock*)?
2. Welcher Prozess darf als erster in den kritischen Abschnitt eintreten, wenn Prozess P_1 den Schreibzugriff auf `turn` einen Nanosekunden-Bruchteil nach Prozess P_0 ausführt?
3. Begründen Sie die Entscheidung anhand der booleschen Schleifenbedingungen.

Simulator ↗

3.) Fehleranalyse: Weglassen der Höflichkeitsvariable `turn`

Ein Entwickler möchte den Code vereinfachen und streicht die Zuweisung von `turn`:

Fehlerhafter Eintritt P_0	**Fehlerhafter Eintritt P_1**
`interested[0] = true;`	`interested[1] = true;`
`while (interested[1]); // warten`	`while (interested[0]); // warten`

Aufgaben:

1. Zu welchem schwerwiegenden Synchronisationsfehler führt dieser fehlerhafte Code?
2. Konstruieren Sie ein konkretes zeitliches Interleaving zweier Prozesse, bei dem dieser Fehler auftritt.
3. Welche der vier Tanenbaum-Bedingungen für kritische Abschnitte wird dadurch verletzt?

Simulator ↗

4.) Fehleranalyse: Vertauschte Reihenfolge von Zuweisungen

Ein Programmierer tauscht in `enter_region` versehentlich die Zuweisungen um:

Vertauschter Eintritt P_0	**Vertauschter Eintritt P_1**
`turn = 0; // Erst turn setzen!`	`turn = 1; // Erst turn setzen!`
`interested[0] = true; // Dann Flag!`	`interested[1] = true; // Dann Flag!`
`while (interested[1] && turn == 0);`	`while (interested[0] && turn == 1);`

Aufgaben:

1. Zeigen Sie durch ein konkretes Interleaving, dass dadurch der **wechselseitige Ausschluss verletzt** wird!
2. Beschreiben Sie den Zustand beider Prozesse am Ende Ihres Interleavings.

Simulator ↗

5.) Realität moderner Multicore-Hardware & Memory Barriers

In der Theorie gilt der Peterson-Algorithmus als mathematisch bewiesen korrekt. Auf modernen Mehrkernprozessoren (z. B. x86, ARM oder Apple Silicon) führt eine naive C-Implementierung jedoch im praktischen Stresstest reproduzierbar zu Race Conditions.

Aufgaben:

1. Welcher Hardware-Mechanismus moderner CPUs (bzw. Optimierung moderner Compiler) verursacht dieses Phänomen?
2. Wie muss der Code auf modernen Plattformen geschützt werden, um die Korrektheit wiederherzustellen?